

Policy Pilot V2 — Technical Design Summary

1. Definition of Live Learning

Policy Pilot defines **live learning** as converting reviewed access decisions into governed organizational knowledge that can influence future recommendations without model retraining or a new code or prompt deployment. The model and versioned **v6** prompt remain unchanged. Learning occurs through database evidence: governance-approved precedents and operating rules are retrieved for later requests. The lifecycle is:

Human review → proposed knowledge → governance approval → active retrieval → future influence → revocation.

The AI only recommends **APPROVE**, **DENY**, or **ESCALATE**. A human remains accountable for the final **GRANT** or **DENY**.

2. Historical Decisions and Retrieval

Each review preserves the final decision, reviewer identity and timestamp, agreement with the model, deviation from the effective recommendation, required reason code, override rationale, missing context, and whether the case may be reusable. A reusable decision becomes a **PROPOSED** precedent. Its deterministic summary preserves the request, model recommendation, human outcome and reviewer feedback. It also stores the policy versions in effect when the decision was made.

Similarity does not embed the long review narrative. Historical and new requests use the same canonical representation:

- Request type, system and entitlement.
- Requester title, department and cost center.
- Business justification.

This prevents asymmetric comparison between a detailed historical narrative and a concise new request. Embedding versions also prevent old vector formats from entering current retrieval. A precedent reaches the model only when it is **ACTIVE**, matches the same system and entitlement, uses the current embedding version, meets the **0.75** cosine-similarity floor and is not stale under current policy versions. Proposed, revoked, stale or insufficiently similar precedents are excluded.

3. Applicability and Conflict Handling

Similarity identifies candidates; it does not prove applicability. The model must review every retrieved precedent through **precedent_review**, decide whether its context applies and cite applicable cases separately from formal policy with a relevant explanation. Formal policy remains the primary authority. Precedents and operating rules are supporting evidence and cannot independently justify approval or denial.

A deterministic grounding gate verifies that:

- Every retrieved precedent and operating rule was reviewed exactly once.
- No fabricated or duplicate IDs were introduced.
- Cited precedents were actually retrieved.
- Applicable precedent outcomes do not contradict the recommendation without escalation.
- Selected policy source IDs came from evidence retrieved for that request.
- Final policy citations are copied from authoritative evidence rather than written by the model.

The result becomes **ESCALATE** when evidence is omitted, policy is insufficient, applicable precedents disagree, policy conflicts with precedent or operating guidance, or the recommendation cannot be grounded safely. Semantic applicability remains a model judgment, but the gate prevents silent omission of inconvenient evidence.

4. Proposed Operating Rules

Pattern discovery runs on a schedule and can also be triggered through the governance UI. It groups reviews by system and entitlement, with a demo-scale minimum of two decisions.

It currently detects:

- **Recurring exceptions:** at least 50% of decisions overrode the recommendation.
- **Inconsistent decisions:** the group contains both grants and denials.

A detected pattern becomes a **PROPOSED** rule suggestion with supporting decision IDs. It has no effect until reviewed.

A governance approver may dismiss it, accept it without operational guidance, or accept it with guidance. Acceptance with guidance creates an exact-scope **ACTIVE** operating rule that is retrieved for subsequent matching requests. Only recurring-exception acceptance may activate eligible supporting precedents; inconsistency and outdated-policy warnings do not automatically make their supporting cases authoritative.

5. Governance and Rollback

The original reviewer cannot activate their own precedent. A separate **GOVERNANCE_APPROVER** must promote it from **PROPOSED** to **ACTIVE**. Governance transitions use guarded updates and append-only audit records containing the actor, timestamp and state change.

Active precedents and operating rules can be revoked. Retrieval reads only current **ACTIVE** records, so revocation affects the next request without deployment. Historical recommendations remain unchanged for auditability.

6. Duplicate Decision and Execution Prevention

Ingestion is protected by an **Idempotency-Key**, a unique external **request_id** and database constraints.

Only one final human decision may exist per request. A conditional **RECOMMENDED** → **DECIDED** transition and unique decision constraint prevent concurrent reviewers from both succeeding.

The human decision, structured feedback and **PENDING** execution-outbox record are committed in one transaction. Execution then occurs through a separate BullMQ worker using:

- One execution record per decision.
- The execution ID as the BullMQ job ID and downstream idempotency key.
- Compare-and-swap claims, expiring leases and fencing tokens.
- Recovery sweeps for lost jobs and expired leases.
- Retry and backoff for transient failures.

Delivery is at-least-once. Production exactly-once effects require the downstream system to durably honor the idempotency key.

7. Assumptions, Limitations and Trade-offs

The **0.75** threshold is a conservative demo value requiring calibration on a larger labelled corpus. Pattern discovery groups by system and entitlement rather than proving semantic similarity.

Staleness is conservative: an unrelated policy change can invalidate a precedent, while a newly added policy may not be recognized as a dependency. Multiple active operating rules have no priority or supersession relationship, and revoked knowledge is not retroactively flagged in historical recommendations.

Applicability remains probabilistic, making the deterministic completeness gate essential. Authentication, manager-approval verification and downstream provisioning are simulated. Remaining production gaps include an LLM timeout, durable external execution integration, rule supersession and frontend automated tests.